Sprint 5

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Grupo 2:

Alejandro Alcázar Mendoza
Sergio Fernández Cordero
Liam Esgueva González

Índice

1. Introducción **3**
1.1. Contexto . . . 3
1.2. Descripción del problema y objetivos del sprint . . . 3
1.3. Justificación . . . 3

2. Metodología **4**
2.1. Entorno de desarrollo . . . 4
2.2. Metodología SCRUM . . . 4
2.3. Diseño de la solución . . . 5
2.3.1. Broker Kafka en Amazon EC2 . . . 5
2.3.2. Productor y consumidor básicos . . . 5
2.3.3. Conexión en tiempo real con la API de Binance . . . 5

3. Resultados **7**
3.1. Descripción . . . 7
3.2. Ejecución . . . 7
3.2.1. Validación del broker y flujo básico . . . 7
3.2.2. Streaming de datos desde Binance . . . 8
3.3. Discusión de los resultados . . . 8
3.3.1. Flujo básico productor–consumidor . . . 8
3.3.2. Productor Binance . . . 9
3.3.3. Consumidor Binance . . . 9

4. Conclusión **9**
4.1. Resumen del proceso . . . 9
4.2. Principales logros . . . 10

Sprint 5

1. Introducción

1.1. Contexto

El presente proyecto se desarrolla en el marco de la asignatura Tecnologías de Procesamiento Big Data y tiene como objetivo el diseño e implementación de una arquitectura Big Data completa orientada al análisis de datos de trading de criptomonedas. El proyecto simula un entorno real en el que distintas consultoras tecnológicas trabajan para un gran banco que desea crear un nuevo departamento especializado en el asesoramiento a clientes en trading de criptoactivos.

La solución propuesta se basa en tecnologías de **Amazon Web Services (AWS)** y contempla tanto el procesamiento de datos históricos en modo batch como la ingesta y análisis de datos en tiempo real (streaming). Para la gestión del proyecto se adopta la metodología ágil Scrum, estructurando el desarrollo en sprints incrementales y entregables funcionales al final de cada iteración.

Este documento recoge la planificación y los objetivos correspondientes al **quinto sprint del proyecto**.

1.2. Descripción del problema y objetivos del sprint

El objetivo de este sprint es implementar una canalización de datos en tiempo real para la ingesta de cotizaciones de criptomonedas mediante **Apache Kafka**. Para ello, se despliega un broker Kafka sobre una instancia **Amazon EC2** y se desarrollan un **productor** y un **consumidor** que permiten transmitir y recibir datos de mercado en streaming desde la API de **Binance**.

Concretamente, se deben implementar: un productor básico de prueba, un consumidor básico de prueba, un productor que conecte con la API de Binance vía WebSocket para capturar velas de un minuto de **ADA/USDT** y las publique en un topic Kafka, y un consumidor que lea y muestre esos mensajes en tiempo real.

1.3. Justificación

El uso de **Apache Kafka** como plataforma de mensajería distribuida permite gestionar flujos de datos de alta frecuencia con baja latencia, lo que resulta especialmente adecuado para la captura de cotizaciones de criptomonedas en tiempo real. La integración con la API de **Binance** a través de WebSocket garantiza la recepción inmediata de datos de mercado actualizados minuto a minuto.

Desplegar el broker sobre **Amazon EC2** proporciona flexibilidad y escalabilidad, permitiendo ajustar los recursos del clúster según la carga de trabajo. Esta arquitectura constituye la base sobre la que se construirán las etapas de procesamiento y análisis en tiempo real previstas en sprints posteriores.



Figura 1. Milestone 5

2. Metodología

2.1. Entorno de desarrollo

El entorno de este sprint se basa en un broker **Kafka** desplegado sobre una instancia**Amazon EC2**. En dicha instancia se instalaron las dependencias necesarias para la ejecución del proyecto, incluyendo **Java**, **Apache Kafka** y las librerías necesarias para la conexión con **Binance**, permitiendo así la ingesta de datos en tiempo real desde el exchange hacia Kafka. Para la ingesta de datos de mercado se utiliza la librería `python-binance`, que proporciona acceso a la API de Binance, incluyendo soporte para conexiones WebSocket mediante el `ThreadedWebsocketManager`. Los mensajes producidos y consumidos se serializan en formato **JSON**.

2.2. Metodología SCRUM

Para seguir la metodología SCRUM, se analizó la historia de usuario correspondiente al sprint actual:

Historia de Usuario (HU-6)

6.1. Como:
Manager del proyecto.

6.2. Quiero:
Que las consultoras:

- Despliegen un **broker Kafka** sobre una instancia **Amazon EC2**.
- Implementen un **productor y consumidor básicos** para verificar el correcto funcionamiento del broker.
- Desarrollen un **productor Kafka** que se conecte a la API de **Binance** mediante WebSocket y capture velas de un minuto de **ADA/USDT**, publicando los datos en tiempo real en un topic Kafka.
- Desarrollen un **consumidor Kafka** que lea los mensajes del topic y los procese en tiempo real.

6.3. Para:
Establecer la **canalización de datos en streaming** que servirá de base para el análisis de cotizaciones en tiempo real en sprints posteriores.

Descomposición en tareas

Tareas del Sprint

ID	Tarea	Asignado a
#32	Launch VM	Alejandro
#33	Create the producer	Liam
#34	Create the receiver	Liam
#35	Create sprint structure on github	Sergio
#36	Sprint documentarion	Liam y Alejandro
#37	Test producer and consumer	Liam y Alejandro

2.3. Diseño de la solución

La solución se articula en los componentes que se describen a continuación.

2.3.1. Broker Kafka en Amazon EC2

Para operar con el broker desde un cliente local es necesario instalar **Java** y los binarios de Kafka, y configurar un archivo `client.properties` con los parámetros de autenticación. A través de los scripts de administración de Kafka es posible listar, crear, describir y eliminar topics, así como inspeccionar el estado de los consumer groups.

2.3.2. Productor y consumidor básicos

Como paso previo a la integración con Binance, se implementan un **productor básico** y un **consumidor básico** para verificar el correcto funcionamiento del broker y familiarizarse con la API de `kafka-python`, como se muestra en la Figura 5.

2.3.3. Conexión en tiempo real con la API de Binance

Para la captura de datos de mercado en tiempo real se utiliza la librería `python-binance`, que proporciona el `ThreadedWebsocketManager`. Este componente gestiona la conexión WebSocket con Binance en un hilo independiente, permitiendo que el programa principal siga ejecutándose mientras se reciben eventos de mercado de forma asíncrona. Se han generado dos scripts de Python, uno para el productor y otro para el consumidor. El método más importante del productor es el callback:

Listing 1. Envío de datos en tiempo real desde Binance

```
def handle_kline(msg):
    k = msg.get("k")
    if k is None:
        return
    # Just closed
    if not k["x"]:
        return
    close_time_ms = k["T"]
    timestamp_str = datetime.utcfromtimestamp(close_time_ms / 1000).strftime(
        "%Y-%m-%dT%H:%M:%SZ"
    )
    symbol = k["s"]
    key = symbol.replace("USDT", "").replace("USD", "")

    message = {
        "symbol": symbol,
        "@timestamp": "@"+timestamp_str,
        "close": float(k["c"]),
        "volume": float(k["v"]),
    }

    print(
        f"[VELA CERRADA] {symbol} | {timestamp_str} | "
        f"close={message['close']} | vol={message['volume']}"
    )

```

```
    producer.send(
        topic=TOPIC,
        key=key,
        value=message,
        timestamp_ms=close_time_ms,
    )
    producer.flush()

    print(
        f"[KAFKA] Publicado en topic {TOPIC} | key={key} | timestamp_ms={
            close_time_ms}"
    )
```

El consumidor recibe el mensaje en formato json.

Listing 2. Envío de datos en tiempo real desde Binance

```
print(f"Escuchando topic '{TOPIC}' en {BOOTSTRAP_SERVERS}...")
print("Ctrl+C para salir\n")

try:
    for record in consumer:
        print(
            f"[{record.value.get('@timestamp')}] "
            f"key={record.key} | "
            f"close={record.value.get('close')} | "
            f"volume={record.value.get('volume')} | "
            f"offset={record.offset}"
        )
except KeyboardInterrupt:
    pass
finally:
    consumer.close()
    print("Conexión cerrada.")
```

3. Resultados

3.1. Descripción

En esta sección se presentan los resultados obtenidos tras la ejecución de los distintos componentes implementados durante el sprint. Se muestra el correcto funcionamiento del broker Kafka, la validación del flujo básico productor–consumidor y la recepción y publicación de datos de mercado en tiempo real desde Binance.

3.2. Ejecución

3.2.1. Validación del broker y flujo básico

En primer lugar se verificó la conectividad con el broker Kafka mediante la creación del topic **imat3a-ADA**, el listado de topics disponibles y la descripción del topic creado.



Figura 2. Creación del topic

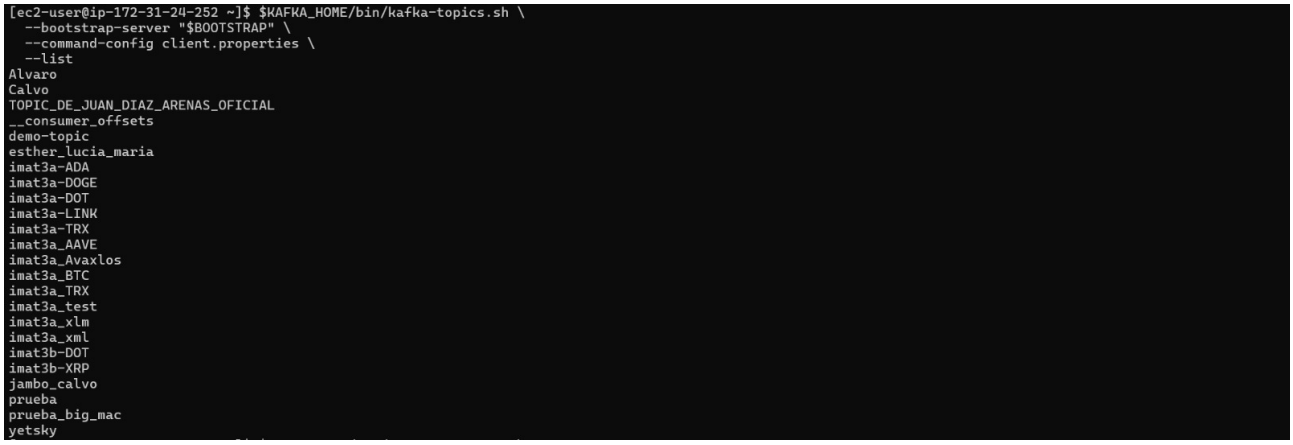


Figura 3. Listado de topics

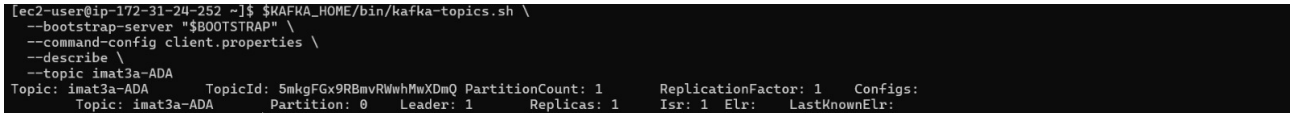


Figura 4. Descripción del topic

A continuación se ejecutó el productor básico, que publicó un mensaje de prueba en el topic `imat3a-ADA`, y el consumidor básico, que lo leyó y mostró por pantalla.

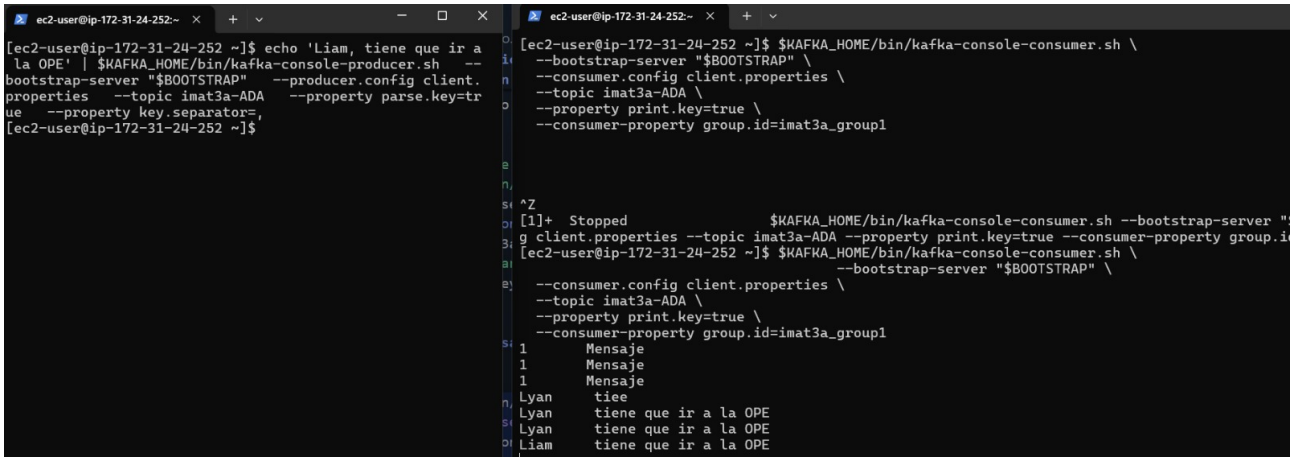


Figura 5. Envío manual

3.2.2. Streaming de datos desde Binance

Una vez validado el broker, se ejecutaron en paralelo el productor y el consumidor de Binance mediante los sctips en la EC2.

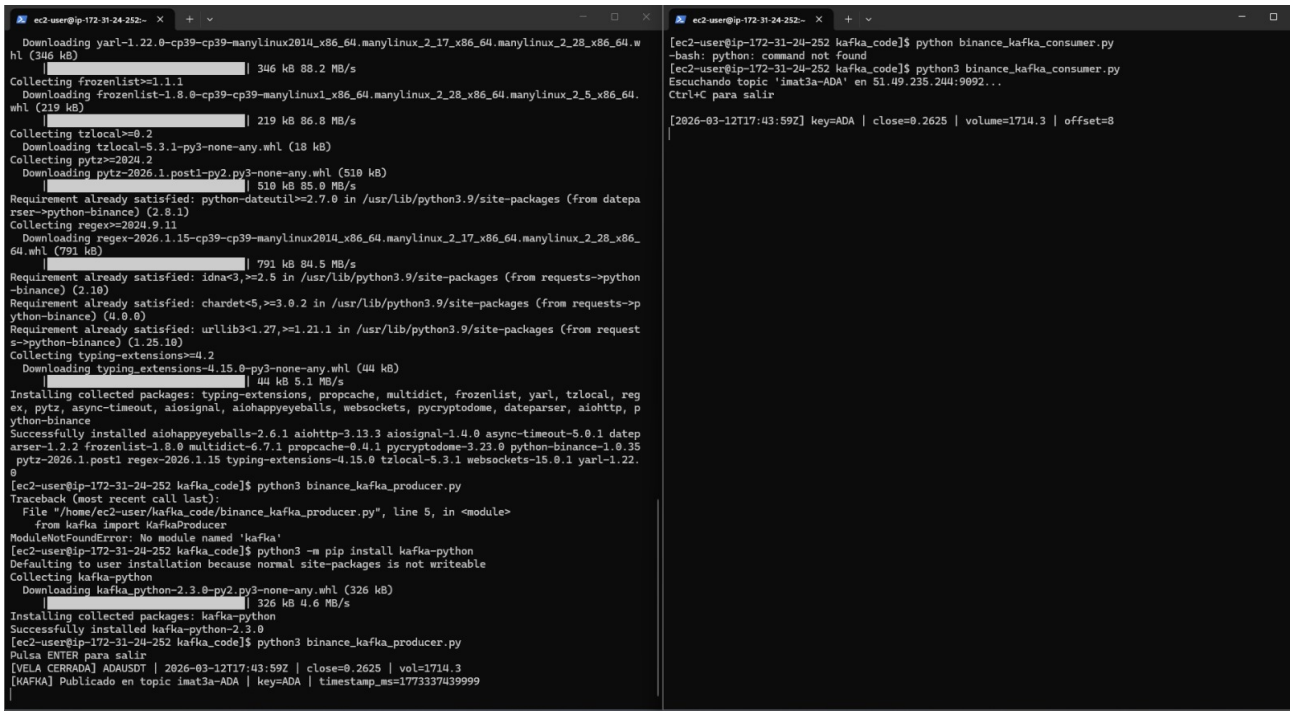


Figura 6. Descripción del topic

3.3. Discusión de los resultados

3.3.1. Flujo básico productor–consumidor

La ejecución del productor y consumidor básicos confirmó que el broker Kafka desplegado en la instancia EC2 sea accesible y funcione correctamente. Los mensajes publicados en el topic `imat3a-ADA` fueron recibidos por el consumidor con la clave, el valor, el offset y el timestamp esperados, validando el pipeline completo de extremo a extremo.

3.3.2. Productor Binance

El productor basado en WebSocket demostró ser capaz de mantenerse conectado a la API de Binance de forma continua, recibiendo eventos de vela en tiempo real. El filtrado de velas cerradas (`k["x"] == True`) garantiza que únicamente se publican en Kafka datos completos y definitivos, evitando registros parciales que podrían distorsionar el análisis posterior. El uso del timestamp de cierre de la vela como marca temporal del mensaje en Kafka asegura la ordenación temporal correcta de los registros en el topic.

3.3.3. Consumidor Binance

El consumidor configurado demostró recibir los mensajes de forma inmediata tras su publicación, con una latencia mínima observable entre la publicación y la recepción. Los campos `@timestamp`, `close` y `volume` se deserializaron correctamente desde el formato JSON, confirmando que la serialización y deserialización de mensajes funciona de forma consistente entre productor y consumidor.

La arquitectura implementada en este sprint establece una base sólida para las etapas de procesamiento en tiempo real que se desarrollarán en sprints posteriores, permitiendo alimentar pipelines de análisis técnico con datos de mercado actualizados minuto a minuto.

4. Conclusión

4.1. Resumen del proceso

En este sprint se ha implementado una canalización de datos en tiempo real basada en **Apache Kafka**, desplegado sobre una instancia **Amazon EC2**. Se ha desarrollado tanto un flujo básico de validación (productor y consumidor de prueba) como la integración completa con la API de **Binance** mediante WebSocket, capturando velas de un minuto de **ADA/USDT** y publicándolas en un topic Kafka. Un consumidor en tiempo real lee y procesa los mensajes a medida que son publicados. Las tareas se han repartido siguiendo la metodología **SCRUM**.

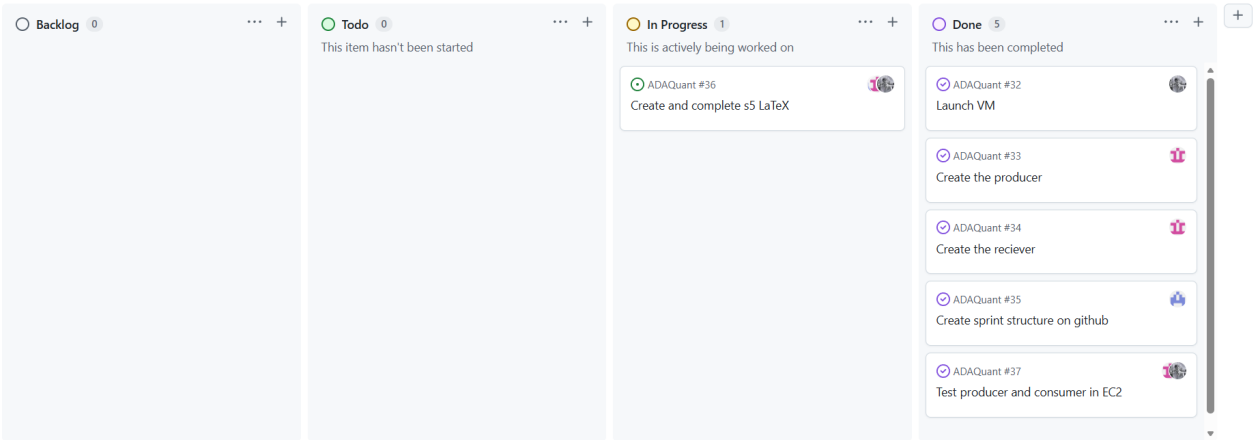


Figura 7. Git board

4.2. Principales logros

- Validación del **pipeline end-to-end** mediante un productor y consumidor básicos sobre el topic de prueba.
- Integración con la **API de Binance** a través de WebSocket para la captura de cotizaciones de **ADA/USDT** en tiempo real con intervalo de un minuto.
- Publicación de velas cerradas en **Kafka** con serialización JSON y marca temporal correcta, garantizando la integridad y el orden de los datos.
- Consumo en tiempo real de los mensajes publicados con mínima latencia observable, sentando las bases para el procesamiento en tiempo real en sprints futuros.